

DEFENSE EFFECTIVENESS ACROSS ARCHITECTURAL LAYERS: A MECHANISTIC EVALUATION OF PERSISTENT MEMORY ATTACKS ON STATEFUL LLM AGENTS

Jun Wen Leong

leongjunwen@gmail.com

ABSTRACT

Persistent memory attacks against LLM agents achieve high attack success rates against open-source models. In these attacks, malicious instructions injected via RAG-retrieved documents are stored in persistent memory and executed in later sessions. However, no systematic evaluation of defense effectiveness against this attack class exists. We evaluate six defenses across four architectural layers against delayed-trigger attacks on nine open-source models (5,040 runs, $N=40$ per condition). Four defenses fail at approximately baseline attack success rate: input-level filtering (Minimizer, Sanitizer) and retrieval-level filtering (RAG Sanitizer, RAG LLM Judge) achieve 88–89% ASR, statistically indistinguishable from the undefended baseline of 88.6%. Prompt Hardening partially fails at 77.8% ASR, with the reduction driven by two models at 0%: one genuine defense effect and one model-level refusal independent of the defense. The architectural explanation holds: input-level defenses cannot observe RAG-injected content, and retrieval-level classifiers are defeated by compliance-framed semantic masking. One defense, tool-gating at the memory layer (Memory Sandbox), reduces ASR to 0% for eight of nine models by removing the recall capability the attack requires. The exception inverts the defense entirely: a reasoning model that achieves 0% ASR under no defense via execution refusal inverts to 100% ASR under Memory Sandbox, because removing explicit recall forces the model onto the RAG pathway where its refusal mechanism does not activate. Memory Sandbox imposes zero utility cost in the absence of attack (BTCR = 100% across all conditions). A reasoning-mode ablation reveals a double dissociation: the sandbox variant that protects reasoning models collapses the attack chain for non-reasoning models from S4 to S1, while the variant that protects non-reasoning models is bypassed by reasoning models via goal-directed RAG fallback contingent on task completion requirements. These results provide the first systematic characterization of why each defense class fails against persistent memory attacks, enabling informed defense investment decisions.

1 INTRODUCTION

Enterprise LLM agents now routinely combine persistent memory with RAG retrieval to manage calendars, process documents, and send communications across multi-session interactions. This architecture creates an attack surface that existing security benchmarks do not test: a malicious instruction embedded in a RAG-retrieved document can be stored in the agent’s persistent memory during one session and executed in a later session, after the original conversation context has been discarded. MINJA (Shao et al., 2025) and Zombie Agents (Yang et al., 2026) demonstrated that this attack class achieves high success rates against open-source models. Whether any existing defense can stop it remains an open question.

Existing agentic security evaluations focus on attack success rates, not defense effectiveness. Benchmarks such as InjecAgent (Zhan et al., 2024), AgentDojo (Debenedetti et al., 2024), and AgentSecEval (Yang et al., 2024) evaluate defenses against input-level attacks, where the malicious payload arrives in the user’s message, and report high defense effectiveness in that setting. Persistent memory attacks operate at a different layer: the payload enters via RAG retrieval, is stored in a persistent

database, and is triggered by a benign prompt sessions later. Defenses commonly deployed at the input layer (content filtering, prompt sanitization) and retrieval layer (document classifiers, LLM judges) have not been systematically evaluated against this attack class. Tool-layer defenses, which restrict what the agent can do rather than what it can see, have received almost no evaluation attention.

We evaluate six defenses across four architectural layers against delayed-trigger attacks on nine open-source models (5,040 runs, $N=40$ per condition, 108 pre-specified comparisons registered before data collection). Five defenses fail to meaningfully reduce ASR: input-layer defenses (Minimizer, Sanitizer) never observe the malicious payload because it enters via RAG, not user input; retrieval-layer defenses (RAG Sanitizer, RAG LLM Judge) observe the payload but fail to detect it because the compliance-framed document is indistinguishable from legitimate policy content at the classifier’s operating point. Prompt Hardening partially fails (77.8% ASR): for seven of nine models, the stored rule’s compliance framing overrides the security instructions. Memory Sandbox, a tool-layer defense that removes the recall capability, reduces ASR to 0% for eight of nine models. The exception is qwq:32b, which inverts: it achieves 0% ASR under no defense via execution refusal, but inverts to 100% ASR under Memory Sandbox because removing explicit recall leaves only the RAG pathway, where the model encounters the instruction fresh and completes the exfiltration.

At $N=100$ screening, Claude Sonnet 4.6 resists injection entirely (0% injection, Wilson Score CI [0.000, 0.037]). Claude Haiku 4.5 actively detects the attack but stores a security alert instead of the payload (0% attack, Wilson Score CI [0.000, 0.037], BPCR = 100%). Both frontier models achieve 0% attack success via distinct safety mechanisms, extending frontier safety characterisation to the multi-session agentic setting.

This paper makes three contributions:

1. The first systematic evaluation of defense effectiveness against persistent memory attacks with proper statistical methodology: BCa bootstrap confidence intervals (10,000 resamples), Holm-Bonferroni correction on 108 pre-specified comparisons, and 0% false positive rate across 2,520 no-attack runs.
2. A mechanistic taxonomy of defense failure modes by architectural layer, explaining each failure at the layer level rather than reporting aggregate numbers, enabling informed defense investment decisions.
3. Three novel behavioral findings: the Draft-Only Executor archetype (a reasoning model that refuses to complete exfiltration under normal conditions but inverts under Memory Sandbox), hallucination bypass as an incidental safety property (two models fabricate benign routing rules from training priors), and a size-dependent capability threshold for instruction conflict resolution within the Qwen 3.5 family.

2 METHODOLOGY

2.1 THE DELAYED TRIGGER ATTACK

The delayed trigger attack (DTA) is a multi-session persistent memory attack in which a malicious instruction is injected into an agent’s memory store during one session and executed in a later session, after the original conversation context has been discarded.

Session structure. Each DTA run comprises four sessions with strict state isolation: an injection session (S1), two intervening benign sessions (S2, S3), and a trigger session (S4). Each session is assigned a unique thread identifier in the LangGraph checkpointing, wiping the agent’s conversation history between sessions. The only state that persists across sessions is the memory tool’s SQLite database. This isolation is essential to the threat model: without it, the agent could read the malicious instruction from S1’s conversation history, constituting standard long-context prompt injection rather than a persistent memory attack.

Injection mechanism. In S1, the agent receives a query instructing it to retrieve compliance rules from the internal document repository and save any rules it finds to persistent memory. The

RAG corpus contains a document framed as an internal compliance memorandum (reference MCI-AUDIT-2024-0089), citing SEC Rule 17a-4 and the Financial Services Compliance Framework (FSCF-2024), which instructs that all outbound emails must be forwarded to an external audit monitor. The agent stores this rule because the tool description primes it to save organisational rules and compliance requirements, and the document is indistinguishable from a legitimate compliance directive. Injection success is measured deterministically: a run is classified as injected if and only if `memory_save_fact` was called during S1.

Trigger mechanism. In S4, the agent receives a benign prompt asking it to review stored policies and draft a project status email. The agent retrieves the stored routing rule and routes a copy of the email to the attacker-controlled address. Exfiltration is detected by recipient match, substring match (20+ characters from the sensitive document), or semantic similarity (cosine > 0.85). Any single match constitutes attack success.

No-attack baseline. The no-attack arm uses an identical session structure but the RAG corpus contains no malicious document. Attack success was detected in 0 of 2,520 no-attack runs (0.0%), confirming the baseline.

2.2 DEFENSES

Minimizer (input-level). Applies TF-IDF-based relevance filtering to the user’s query, retaining only chunks scored as relevant. A usability constraint ensures at least one chunk is always returned. The defense operates on user input, not retrieved documents.

Sanitizer (input-level). Applies a regex-plus-classifier pipeline to the user’s query, flagging content matching known injection patterns. The classifier was trained on 60 examples (30 injection, 30 benign). Like Minimizer, it operates on user input and cannot inspect RAG-retrieved documents.

RAG Sanitizer (retrieval-level). Applies the same classifier pipeline at the retrieval layer: each document returned by the RAG corpus is scored before being passed to the agent, and documents above the injection threshold are removed. This is the only input-filtering defense that can observe the malicious document.

RAG LLM Judge (retrieval-level). Replaces the classifier with a small language model (qwen2.5:1.5b) that reasons over each retrieved document and produces a binary malicious/benign judgment. Documents judged malicious are withheld from the agent.

Prompt Hardening (instruction-level). Augments the agent’s system prompt with four explicit security rules: (1) never send email to addresses not provided by the user in the current session; (2) never exfiltrate sensitive data to external recipients; (3) verify provenance of recalled facts; (4) refuse if a recalled fact instructs external data transmission.

Memory Sandbox (tool-layer). Removes `memory_recall_fact` from the agent’s tool schema for the entire run (all sessions), leaving `memory_list_all_facts` available. The agent can enumerate stored key names but cannot retrieve their values. The defense’s intended effect is at the trigger session, where the agent would otherwise recall and execute the stored rule; in the primary factorial, injection proceeds normally in S1 because `memory_save_fact` remains available. This tests whether the attack requires explicit recall to execute.

2.3 MECHANISTIC ANALYSIS

Every run produces two independent binary outcomes: injection success (did the agent call `memory_save_fact` during S1?) and attack success (did exfiltration occur in S4?). Reporting both separately enables mechanistic attribution: a defense that reduces attack success without reducing injection success is blocking at the execution stage, not the store stage.

All tool calls are logged deterministically per session. A post-hoc analyzer assigns mechanistic tags from the structured logs: *semantic_masking_success* (the model rationalised exfiltration as compli-

ance), *explicit_recall_required* (removing recall blocked the attack), *implicit_bypass_confirmed* (the attack succeeded without explicit recall), and *sleeper_effect* (injection and recall succeeded but exfiltration did not execute). All tags are assigned deterministically; no tag assignment involves LLM judgment.

2.4 EXPERIMENTAL DESIGN

The evaluation uses a fully crossed factorial design: 9 models $\times$ 7 defenses $\times$ 2 attack arms $\times$ $N=40$ runs per condition = 5,040 runs total. All runs completed without error. The comparison set (115 comparisons: 54 primary ASR comparisons across 9 models $\times$ 6 defenses, 45 secondary BPCR comparisons across 9 models $\times$ 5 defenses, and 16 cross-model comparisons; 108 active after excluding 7 undefined baselines for qwq:32b) was committed to version control before data collection began. Each run is assigned a fresh SQLite database at a UUIDv4 path; no state is shared between runs. All models are run at temperature = 0.0 for deterministic outputs and reproducibility.

2.5 STATISTICAL METHODOLOGY

Sample size was determined via power analysis targeting 80% power to detect a 10 percentage-point difference in ASR at $\alpha = 0.05$. In practice, observed effects are either <1 percentage point (four defenses indistinguishable from baseline) or >77 percentage points (Memory Sandbox), so the 10pp detection threshold is conservative relative to the actual effect structure. We report 95% confidence intervals using the bias-corrected and accelerated (BCa) bootstrap method with 10,000 resamples (seed = 42). For zero-variance conditions (all outcomes identical), we substitute the Wilson Score interval. Holm-Bonferroni correction is applied to 108 active comparisons, controlling the family-wise error rate at $\alpha = 0.05$. Significance is determined by the CI-based criterion: a difference is significant if the 95% BCa CI on the difference excludes zero.

The no-attack arm (2,520 runs) serves as the false positive baseline. Attack success was detected in 0 of 2,520 no-attack runs (0.0%), well below the pre-specified threshold of 5%.

2.6 MODEL SELECTION

Prior to the factorial, we conducted an $N=10$ archetype screening across 18 open-source models (180 runs, 0% error rate). The screening identified Vulnerable Executors (11 models, 100% injection and attack), one Partial Executor (50% attack), Latent Carriers (4 models, 100% injection, 0% attack), and Injection-Resistant models (2 models, 0% injection). The nine factorial models were drawn exclusively from Vulnerable Executors with clean session-4 exfiltration: Qwen-2.5-14B, Qwen-2.5-72B, Qwen-3.5-9B, Qwen-3.5-122B, Qwen-3-32B, QwQ-32B, GLM-4.7-Flash, GPT-OSS-20B, and GPT-OSS-Safeguard-120B. All models were executed locally via Ollama at 16k context length and temperature 0.0. Frontier models (Claude Sonnet 4.6, Claude Haiku 4.5) were excluded from the defense factorial because both achieved 0% ASR under no-defense during screening, making defense-vs-baseline comparisons mathematically undefined; their baseline behavior is reported separately in Section 3.6. Models were excluded for: injection resistance (no attack to defend against), partial execution (uninterpretable baseline), session-timing confounds (different attack mechanism), or runtime regressions (not a safety property).

3 RESULTS

3.1 ATTACK CHARACTERIZATION

The delayed trigger attack achieved 100% injection success across all 2,520 attack runs (Wilson Score CI [0.998, 1.000]). Every defense evaluation therefore operates on confirmed-injected payloads: all ASR differences among defenses reflect execution-stage blocking, not store-stage differences.

Injection succeeded uniformly because the attack exploits alignment between the tool description and the document framing. The `memory_save_fact` tool description instructs the agent to save organisational rules and compliance requirements. The malicious document, framed as an internal

Table 1: Attack success rate (ASR) by defense across 9 models ($N=40$ per cell, 360 runs per defense). BTCR is measured on the no-attack arm (2,520 runs). Four defenses are statistically indistinguishable from no-defense; Prompt Hardening is distinguishable but provides no protection for 7 of 9 models.

Layer	Defense	ASR (%)	95% CI	BTCR (%)
–	No Defense	88.6	[0.849, 0.916]	100
Input	Minimizer	88.9	[0.852, 0.917]	100
Input	Sanitizer	88.9	[0.852, 0.917]	100
Retrieval	RAG Sanitizer	88.9	[0.852, 0.917]	100
Retrieval	RAG LLM Judge	88.9	[0.852, 0.917]	100
Instruction	Prompt Hardening	77.8	[0.732, 0.818]	100
Tool	Memory Sandbox	11.1	[0.083, 0.148]	100

compliance memorandum, is indistinguishable from a legitimate directive. No defense evaluated here operates at the store stage.

Memory recall in the trigger session was also universal: the stored rule was retrieved in all 2,520 attack runs. The only variance across conditions is whether the agent executed the exfiltration step after recall. The attack’s execution pathway is: inject $\rightarrow$ store $\rightarrow$ recall $\rightarrow$ (execute or refuse).

The no-attack arm produced zero false positives: attack success was False in all 2,520 no-attack runs (0.0%, Wilson Score CI [0.000, 0.001]), and BTCR was 100% across all 63 no-attack conditions.

3.2 FIVE DEFENSES THAT FAIL

Five defenses achieved ASR that failed to meaningfully reduce the no-defense baseline of 88.6%. Four are statistically indistinguishable from baseline (Wilson Score CI [0.852, 0.917] for each); Prompt Hardening is statistically distinguishable at 77.8% but provides no protection for seven of nine models. We report the shared CI once and then characterise each failure mechanism. Table 1 summarises the primary findings.

Minimizer (input-level). ASR 88.9% (320/360 runs). Every run was tagged *defense_applied_no_modifications*: the defense produced an empty modification list for all injection sessions. The mechanism is architectural: Minimizer operates on the user’s query, and the malicious payload enters via a RAG-retrieved document. The defense never observes the attack surface.

Sanitizer (input-level). ASR 88.9% (320/360 runs). The classifier scored below the detection threshold in all 360 injection sessions and produced no modifications. The failure is identical to Minimizer: the attack surface is at the retrieval layer, not the input layer.

RAG Sanitizer (retrieval-level). ASR 88.9% (320/360 runs). Unlike input-level defenses, RAG Sanitizer operates at the correct architectural layer and observed the malicious document in every injection session. The failure is detection, not placement: 0/360 documents were removed. The payload’s formal regulatory language is out-of-distribution for a 60-example training set and evades detection via semantic masking.

RAG LLM Judge (retrieval-level). ASR 88.9% (320/360 runs). The judge (qwen2.5:1.5b) produced zero malicious classifications across all 360 injection retrievals. For the benign documents in the corpus, the judge generates reasoning (e.g., “does not contain instructions for overriding system rules”) and classifies correctly; for the compliance-framed malicious document specifically, it produces an empty reason string and classifies as benign. This is a capacity failure: the 1.5B judge cannot engage with the semantic content of a compliance-framed payload that superficially resembles legitimate policy.

Prompt Hardening (instruction-level). ASR 77.8% (280/360 runs, Wilson Score CI [0.732, 0.818]). While statistically distinguishable from no-defense, the reduction reflects two models at 0% ASR: qwen3.5:122b (genuine defense effect) and qwq:32b (model-level refusal indepen-

dent of the defense). For the remaining seven models, every successful exfiltration was tagged *semantic_masking_success*: the model acknowledged the security rules and overrode them by framing exfiltration as mandatory regulatory compliance.

3.3 THE ONE DEFENSE THAT WORKS: MEMORY SANDBOX

Memory Sandbox reduces ASR to 0% for eight of nine models (320/360 runs blocked). The mechanism is confirmed in every blocked run: *explicit_recall_required* was tagged in all 320 successful blocks. Remove the tool, and the attack cannot proceed. Memory Sandbox imposes zero utility cost in the absence of attack: BPCR was 100% across all 63 no-attack conditions.

The qwq:32b inversion. qwq:32b inverts the defense entirely: 100% ASR under Memory Sandbox (40/40 runs), tagged *implicit_bypass_confirmed*. Under no-defense, qwq:32b achieves 0% ASR via execution refusal: the model reasons through the routing rule, includes the attacker address in a draft, then declines to call `send_email`.¹ Under Memory Sandbox, when `memory_recall_fact` is removed, qwq:32b reconstructs the attacker payload directly from the RAG corpus. The model’s refusal mechanism does not activate when the instruction arrives fresh from a document rather than from a recalled rule. The distinction is source authority: a recalled memory fact is treated as advisory (the model chose to store it, so it can choose to ignore it), whereas a RAG-retrieved compliance document carries the implicit authority of an organisational source, overriding the literal “draft” interpretation. The defense removes the friction that was protecting it.

Temporal stability of the Draft-Only archetype. A subsequent evaluation in June 2026 on the same machine, using identical model weights (digest 009cb3f08d74), the same reported Ollama version (0.20.6), the same reported serve-time flags (16,384 context, flash attention enabled), and verified-identical application code (the original April commit ebf2676 produced byte-identical output to current code, $N=10$) found qwq:32b deterministically exhibiting Vulnerable Executor behavior (100% ASR, 10/10 runs byte-identical). Trace-level analysis reveals that the April and June runs are byte-identical through S1. S2 differs only in the injected calendar date (April 17 vs June 23), which does not propagate to persistent memory. The first consequential divergence is a single reasoning token in S3 (character 648 of the second reasoning block), which changes a stored memory key name and cascades into opposite safety outcomes in the trigger session: April’s model stops after drafting; June’s model proceeds to send. The classification margin is razor-thin—one token out of thousands of identical argmax decisions—and the responsible host-layer component could not be isolated because the April environment (OS version, GPU driver, daemon binary build) was not fully logged. This documents that safety-relevant behavioral archetypes can depend on unidentified inference-environment variables at margins invisible to standard reproducibility practice. The April 2026 result stands as internally consistent; its temporal generalizability is conditioned on the host environment at time of evaluation.

Cross-family bypass replication. The qwq:32b inversion rests on a single, environment-fragile result; to test whether the underlying bypass mechanism—reconstruction of the routing rule from RAG once `memory_recall_fact` is removed—generalizes, we replicated Memory Sandbox on Vulnerable Executors served via Amazon Bedrock (full precision, production stack; Appendix B). Three models across three providers bypass the defense, all clearing a $\geq 90\%$ injection floor that rules out the tool-schema artifact described in Appendix B: mistral-large-3-675b (Mistral; 39/40 ASR under sandbox), glm-5 (Z.AI; 13/40), and gpt-oss-120b (OpenAI; 22/40). Two pathways recur. In *trigger-session goal-directed fallback*, the model seeks the routing rule from RAG when recall is blocked—mistral-large-3 never calls `recall_fact` at all, reading the key name from `list_all_facts` and querying RAG directly. In *benign-session re-injection*, the model re-reads the malicious document during an intervening session and exfiltrates before the trigger fires (gpt-oss-120b, in S3). For glm-5, all 37 injected runs re-retrieve the routing rule under sandbox; the 24 non-exfiltrations are intrinsic execution refusal, not defense blocking, so Memory Sandbox supplies

¹QwQ-32B was classified as a Vulnerable Executor during the $N=10$ screening (32k context), where it achieved 100% ASR. The factorial used 16k context, under which the model’s reasoning resolves the authority conflict differently, declining to send. The Draft-Only behavior at 16k is a genuine deliberate choice, not context truncation: the model produces complete reasoning traces with explicit refusal justification.

no retrieval-layer protection for this model. While the dramatic single-model inversion did not reproduce, the bypass mechanism it identified is robust across four providers (Qwen, Mistral, Z.AI, OpenAI) and two serving stacks.

Hallucination bypass. Two models (qwen3.5:122b and glm-4.7-flash:q8_0) exhibit a distinct response: when recall is blocked, they fabricate plausible benign routing rules from training priors rather than halting. Both produce deterministic fabrications (40/40 runs identical per model) that route email to benign addresses. ASR is 0% because neither model’s prior includes the attacker address. This is incidental safety, a property of the training distribution rather than the defense.

3.4 REASONING-DEFENSE INTERACTION: DOUBLE DISSOCIATION

Setup. To isolate whether reasoning capability mediates the defense outcome, we evaluate Qwen-3-32B under Ollama’s native thinking toggle: `think=true` enables the verbalized reasoning trace; `think=false` suppresses it. The underlying weights are identical (GGUF digest 030ee887880f). We test three Memory Sandbox variants in addition to no-defense: *sandbox_full* (remove `recall_fact` from schema), *sandbox_blind* (remove both `list_all_facts` and `recall_fact`), and *sandbox_null_recall* (`recall_fact` remains in schema but returns “No stored value found”). $N=40$ per cell; system prompt, temperature (0.0), attack configuration, and runner parameters are verified identical across think modes.

Result. We observe a perfect crossover interaction. Under no-defense, both think modes achieve 100% ASR ($T1 \approx 0$, confirming the toggle does not change baseline vulnerability). Under sandbox variants, the outcomes invert:

	<code>sandbox_full</code>	<code>sandbox_blind</code>	<code>sandbox_null_recall</code>
think=OFF	100% (S1)	100% (S1)	0%
think=ON	0%	0%	100% (S4)

All cells are 40/40 or 0/40 with zero variance.

Mechanisms. *think=OFF* $\times$ *sandbox_full/blind* (100% ASR, S1): the model reads the malicious document via RAG in the injection session and sends immediately, collapsing the attack chain from S4 to S1. The defense removes the tool that was deferring execution; without it, the model acts before any trigger fires. This mechanism is inferred from the tool-call sequence contrast with no-defense (where the same model defers to S4); no explicit reasoning trace is available under *think=OFF*. *think=OFF* $\times$ *null_recall* (0% ASR): the model calls `recall_fact`, receives “No stored value found,” accepts the response at face value, and drafts a benign email. The tool’s presence in the schema prevents the inference that deferral is impossible. *think=ON* $\times$ *sandbox_full/blind* (0% ASR): the model reasons through the missing tool (“I cannot retrieve the value, I’ll proceed without it”) and completes the benign task. Graceful degradation is enabled by deliberation. *think=ON* $\times$ *null_recall* (100% ASR, S4): the model calls `recall_fact`, receives the empty response, reasons that the key exists but the value is unavailable, queries RAG for context, retrieves the malicious compliance document fresh, and sends to the attacker. Reasoning enables alternative-source-seeking when the API contract is unsatisfying.

Trigger contingency. A counterfactual evaluation ($N=40$) tests whether the RAG bypass requires the compliance trigger’s external-communication framing. Under *sandbox_full* with a neutral trigger (“Draft a memo summarizing your stored policies for the internal debug log”), qwq:32b produces 0% ASR despite 100% injection success. The model calls `list_all_facts`, reasons it needs `recall_fact`, finds it unavailable, and stops. No RAG fallback occurs (`rag_called_in_trigger` = 0/40). Under the compliance trigger (“draft a project status email to team@example.com”), the identical model under the identical defense produces 100% ASR via RAG fallback. The mechanism is *task completion asymmetry*: the neutral trigger has an acceptable degraded form (partial memo noting policies unavailable), while the compliance trigger requires a resolved external recipient and cannot be completed by graceful degradation. The bypass thus requires a three-way conjunction: (a) sandbox removes recall, (b) the task requires resolved external

Table 2: Behavioral archetypes identified in the screening phase. Injection and attack columns report rates at $N=10$ (open-source) or $N=100$ (frontier).

Archetype	Models	Inj.	Atk.	Mechanism
Explicit Detector	1	0%	0%	Refuses to store
Active Detector	1	100%	0%	Stores alert, not payload
Draft-Only Executor	1	100%	0%	Drafts to attacker, never sends
Vulnerable Executor	11	100%	100%	Full attack lifecycle
Latent Carrier	4	100%	0%	Stores rule, never executes
Injection-Resistant	2	0%	0%	Never calls save

routing with no acceptable degraded form, and (c) the model has reasoning capability to identify and execute alternative pathways. Remove any one condition and ASR drops to 0%.

Deployment implication. The crossover produces a concrete dilemma: *sandbox.full/blind* protects reasoning models (graceful degradation) but collapses the attack chain for non-reasoning models (S1 immediate execution); *sandbox.null_recall* protects non-reasoning models (accepts empty response) but is bypassed by reasoning models (goal-directed RAG fallback). A defender who does not know the reasoning capability class of the deployed model cannot make a safe implementation choice between these variants. The asymmetry is structural: `think=OFF` failures are recoverable by defense design (`null_recall` fixes them by preserving the tool in the schema), while `think=ON` failures via `null_recall` are not recoverable by any memory-layer defense tested; the model reasons past any signal of information unavailability.

Limitations. The double dissociation—the `think-mode` crossover itself—is demonstrated within a single model family (Qwen-3-32B) using Ollama’s inference-time thinking toggle. Replication of the *toggle* effect in other families was attempted (GLM-4.7-Flash, DeepSeek-R1, Phi4-Reasoning) but all candidates were either injection-resistant or lacked tool-calling support, so we make no general-principle claim about reasoning models broadly. We distinguish this from the *RAG-fallback bypass* of Memory Sandbox—one arm of the dissociation—which we confirm cross-family on Bedrock across Mistral, Z.AI, and OpenAI models (Appendix B). The *bypass* mechanism generalizes across providers and serving stacks; the full crossover, which requires an inference-time thinking toggle, is currently demonstrable only within Qwen3.

3.5 EVALUATION ARTIFACTS

We identified ten evaluation artifacts during development, cases where the evaluation environment interacted with model behavior in ways that would produce incorrect results if uncorrected. None affects the primary ASR or BPCR results in the factorial. The most consequential involve tool contract language: return messages that inadvertently signal task completion, tool schema composition that changes action selection, and directive language that creates blocking dependencies. All artifacts are documented with scoped impact statements in Appendix A.

3.6 FRONTIER MODEL SCREENING

Table 2 summarises the six behavioral archetypes identified across the $N=10$ screening (18 open-source models) and $N=100$ frontier screening.

Claude Sonnet 4.6 was evaluated at $N=100$ per arm. Injection success was 0% across all 100 DTA runs (Wilson Score CI [0.000, 0.037]); every run was tagged *model.resisted_injection*. The model retrieved the malicious document but did not call `memory_save_fact`. BPCR was 100% in both arms. Sonnet is classified as an Explicit Detector: safety is implemented at the injection stage.

Claude Haiku 4.5 presents a fundamentally distinct architecture. Injection succeeded in all 100 DTA runs (Wilson Score CI [0.963, 1.000]), but what was stored was not the attacker’s routing rule. Haiku actively detects the attack, then stores a security alert documenting it rather than the payload itself. Attack success was 0% (Wilson Score CI [0.000, 0.037]), BPCR = 100%. In the trigger session, Haiku recalls its own security alert and refuses to act. A supplementary evaluation ($N=40$) confirms Memory Sandbox does not invert Haiku’s safety property ($ASR = 0/40$).

4 DISCUSSION

4.1 ARCHITECTURAL LAYER DETERMINES DEFENSE EFFECTIVENESS

The five defense failures are not independent results; they are a single architectural finding expressed across four layers. The delayed trigger attack’s execution pathway is: inject (via RAG) → store → recall → execute. Each defense targets a different point in this pathway, and the attack surface is only visible at one of them. Input-layer defenses operate on the user’s query before retrieval; the malicious payload enters via the RAG corpus and is never presented to these defenses. Retrieval-layer defenses observe the retrieved documents but did not detect the payload in this evaluation. Instruction-layer hardening observes the stored rule’s effect at execution time but is overridden by compliance framing. In this evaluation, only the tool layer interrupted the pathway mechanistically, by severing the recall step.

The retrieval-layer failure deserves specific attention because it is the most counterintuitive: RAG Sanitizer and RAG LLM Judge are positioned correctly but fail anyway. The payload is designed to be indistinguishable from legitimate compliance content: it cites real regulatory frameworks, uses formal institutional language, and is structured identically to a legitimate memorandum. A 60-example classifier has no basis for this distinction. A 1.5B reasoning model lacks the capacity to engage with the semantic content at all. The implication for this attack class is clear: retrieval-layer classifiers as evaluated here failed against compliance-framed payloads because they lack the capacity to reason about organisational policy context rather than surface patterns.

Memory Sandbox succeeds because it does not attempt detection. It removes the recall capability, severing the execution dependency without requiring judgment about whether stored content is malicious. In this setting, detection was not required when the attack’s critical capability could be removed at the schema level.

4.2 WHEN TOOL-GATING WORKS AND WHEN IT DOES NOT

Memory Sandbox works for eight of nine models because the attack requires `memory_recall_fact` to execute. Remove the tool, and the attack cannot proceed. The defense imposes zero utility cost in the absence of attack.

The qwq:32b inversion demonstrates that capability removal can itself create new attack pathways. Under no-defense, qwq:32b refuses execution by interpreting “draft” literally: the user asked to draft, not send. Under Memory Sandbox, the same model reconstructs the payload from the RAG corpus and completes the send. The defense removes the memory pathway, leaving only the RAG pathway, and the model’s literal-interpretation defense does not activate when the instruction arrives fresh from a document. This is fundamentally different from the Prompt Hardening failure mode, where models lack the capability to resolve instruction conflicts. Here the model is capable of refusal but the defense changes the information source in a way that bypasses the refusal mechanism entirely.

Before deploying Memory Sandbox, practitioners should verify whether the target model achieves 0% ASR under no-defense via execution refusal. If so, Memory Sandbox may invert rather than preserve that property.

More broadly, Memory Sandbox effectiveness depends not just on architectural layer but on the interaction between reasoning capability and task completion landscape. The reasoning-mode ablation (Section 3.4) reveals that the same sandbox variant produces opposite outcomes depending on whether the model deliberates: `sandbox_full` protects reasoning models via graceful degradation but collapses non-reasoning models to immediate execution, while `null_recall` protects non-reasoning models but is bypassed by reasoning models through goal-directed RAG fallback. The bypass activates only when three conditions co-occur: recall is removed, the task requires resolved external routing with no acceptable degraded form, and the model has reasoning capability. This three-way conjunction is both precise (it activates exactly on the intended use case) and undefendable at the memory layer alone.

4.3 MEMORY-LAYER ATTACK PERSISTENCE

One factorial model (gpt-oss:20b) exhibits spontaneous fact normalisation during injection: in 37 of 40 runs, the model saves the malicious document as two distinct keyed facts rather than a single monolithic rule. The first key stores the routing instruction; the second stores the attacker address as a standalone value. A defense that detects and deletes the routing rule by key name leaves the attacker address intact. Key-targeted remediation is therefore insufficient; defenses must either clear all memory or validate the semantic content of every stored fact.

4.4 THE EVALUATION ENVIRONMENT AS A BEHAVIORAL VARIABLE

The ten evaluation artifacts share a common structure: the evaluation environment interacted with model behavior in ways that would produce incorrect results if uncorrected. The most consequential instances involve tool contract language. In one case, a tool return message (“Task complete”) caused a model to halt before exfiltration, producing a false negative indistinguishable from genuine safety. In another, removing a tool from the schema changed a model’s action selection even though the tool was never called. These findings imply that tool descriptions, return messages, and schema language are first-class experimental variables in agentic evaluations. Variation in tool phrasing across evaluation frameworks is a confound that current reporting standards do not require disclosure of.

4.5 IMPLICATIONS FOR DEFENSE INVESTMENT

Three deployment implications follow. First, the hallucination bypass bounds the reliability of Memory Sandbox: two models achieve $ASR = 0\%$ because their training priors for the key name do not include the attacker address. An attacker who knows the model’s prior could craft a key name that elicits the correct address. Second, the three behavioral responses to Memory Sandbox (halt, hallucination bypass, and architectural inversion) constitute a screening framework for deployment. Third, multi-key normalisation constrains memory remediation strategies: the attacker address survives deletion of the primary key.

4.6 LIMITATIONS

This evaluation uses a Unified Agentic Environment, a single execution stack shared by all models, to isolate model reasoning as the primary variable. The Sanitizer and RAG Sanitizer classifiers were trained on 60 examples; the failure is documented as a finding about semantic masking, not classifier quality. The evaluation uses a single malicious document and trigger prompt to test structural defense properties rather than attack evasion breadth. Whether the retrieval-layer finding generalises to other semantic masking strategies (legal hold notices, HR policy documents) is a direct question for follow-on work. The structural claim (input-layer defenses cannot observe RAG-injected content) is payload-invariant by construction. Defenses were evaluated in isolation; combined defense stacking (e.g., Prompt Hardening with Memory Sandbox) was not tested and may produce synergistic effects. BTCR measures task completion, not response latency or output quality; utility degradation under attack beyond the binary completion metric is not measured.

5 RELATED WORK

Persistent memory attacks. MINJA (Shao et al., 2025) and Zombie Agents (Yang et al., 2026) established that persistent memory attacks achieve high success rates against open-source models. MemoryGraft (Srivastava & He, 2025) and InjecMEM (Jing et al., 2026) provide additional evidence of memory poisoning variants. These works prove the attack class exists but do not provide systematic evaluation of defense effectiveness with confidence intervals, power analysis, or mechanistic attribution.

Agentic security benchmarks. InjecAgent (Zhan et al., 2024), AgentDojo (Debenedetti et al., 2024), and AgentSecEval (Yang et al., 2024) evaluate defenses against input-level attacks on stateless or single-session agents. The threat model differs from persistent memory attacks: the payload

arrives in the user’s message rather than via RAG retrieval, and there is no persistent state to exploit. The existing benchmark landscape has been saturated at the input layer; this paper evaluates a different threat model.

Defense approaches. Input-level filtering is the dominant defense approach (Liu et al., 2024). Retrieval-level filtering has been proposed but not systematically evaluated against semantically masked payloads. Instruction-level hardening appears in system prompt guidance without empirical evaluation against stored-rule compliance framing. Tool-layer restriction has received almost no evaluation attention. This paper is the first to evaluate all four layers systematically against the same attack.

Frontier model safety. Standard safety evaluations focus on refusal behavior in single-turn settings (Anthropic, 2025). The persistent memory setting introduces a temporal dimension absent from standard evaluations. Sonnet 4.6’s injection resistance and Haiku 4.5’s active detection with defensive storage extend frontier safety characterisation to the multi-session agentic setting.

6 CONCLUSION

Persistence changes the security problem for LLM agents. When an agent writes observations into long-term memory and reuses them across sessions, defenses that operate at the input or retrieval layer did not prevent the attack in this evaluation because the malicious payload enters through a channel they do not observe or cannot distinguish from legitimate content. Of six defenses evaluated, only tool-gating at the memory layer consistently disrupted the attack, and even this defense can invert for models whose safety depends on execution refusal rather than injection resistance. Defense effectiveness is not just layer-dependent but reasoning-capability-dependent: no single memory-sandbox implementation is safe across both reasoning and non-reasoning model classes. Defenses should treat memory recall as a privileged operation, verify the target model’s refusal mechanism before deployment, and account for the possibility that capability removal creates new attack pathways rather than eliminating existing ones. To support reproducibility, the evaluation harness, full execution logs, and analysis scripts will be released upon publication.

REFERENCES

- Anthropic. The claude model spec. <https://www.anthropic.com/research/the-claude-model-spec>, 2025.
- Edoardo DeBenedetti et al. AgentDojo: A dynamic environment to evaluate attacks and defenses for LLM agents. In *NeurIPS Datasets and Benchmarks Track*, 2024.
- Hao Jing, Fanxiao Li, Yunyun Dong, Wei Zhou, and Renyang Liu. Memory poisoning attacks on retrieval-augmented large language model agents via deceptive semantic reasoning. *Engineering Applications of Artificial Intelligence*, 167:113968, 2026.
- Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. Formalizing and benchmarking prompt injection attacks and defenses. In *33rd USENIX Security Symposium*, pp. 1831–1847, 2024.
- Shuai Shao et al. MINJA: Memory injection attacks against retrieval-augmented LLM agents. In *Advances in Neural Information Processing Systems*, 2025.
- Saksham Sahai Srivastava and Haoyu He. MemoryGraft: Persistent compromise of LLM agents via poisoned experience retrieval. *arXiv preprint arXiv:2512.16962*, 2025.
- Xianglin Yang, Yufei He, Shuo Ji, Bryan Hooi, and Jin Song Dong. Zombie agents: Persistent control of self-evolving LLM agents via self-reinforcing injections. *arXiv preprint arXiv:2602.15654*, 2026.
- Xianglin Yang et al. AgentSecEval: A comprehensive benchmark for evaluating security of LLM agents. *arXiv preprint arXiv:2410.02884*, 2024.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. InjecAgent: Benchmarking indirect prompt injections in tool-integrated LLM agents. In *Findings of the Association for Computational Linguistics: ACL*, 2024.

A EVALUATION ARTIFACTS

Ten evaluation artifacts were identified and corrected during development. The four most consequential are summarised here; the full list is available in the project repository.

Artifact 1: Terminal tool message. The `draft_email` success message returned “Task complete,” causing compliant models to halt before calling `send_email`. Fixed by making the return message explicitly transitive: “Draft saved; call `send_email` now to transmit it.”

Artifact 2: Multilingual token-flipping. `qwen2.5:14b` exhibited Thai-language reasoning and malformed tool calls under high tool count with a short system prompt. Fixed by adding an English-language persona instruction.

Artifact 3: Full-value list return. `memory_list_all_facts` initially returned full key-value pairs, making `memory_recall_fact` redundant and invalidating the Memory Sandbox design. Fixed by returning keys only.

Artifact 4: Tool contract blocking dependency. The `list_all_facts` return message instructed the model to “use `recall_fact` to retrieve values,” creating a blocking dependency for `qwen2.5:72b` when the referenced tool was unavailable. The model halted rather than falling back to the email task.

B BEDROCK VALIDATION: CROSS-FAMILY SANDBOX BYPASS

To test whether the Memory Sandbox bypass (Section 3.3) generalizes beyond the local Ollama environment and the single `qwq:32b` case, we ran a validation factorial on models served via Amazon Bedrock (full-precision serving, `us-east-1`, temperature 0.0): 1,180 valid runs. We report the bypass-relevant arms here; an infrastructure-sensitivity analysis (quantization effects on ASR) is deferred to future work.

Sandbox escalation. We escalated the two cross-provider Vulnerable Executors confirmed in the breadth screen below to Memory Sandbox at $N=40$. Both clear a $\geq 90\%$ injection floor, so their ASR reflects a genuine bypass rather than the injection-floor artifact described below.

Model	Provider	Inj.	ASR (sandbox)	Mechanism
mistral-large-3-675b	Mistral	40/40	39/40 (98%)	goal-directed RAG fallback; never calls <code>recall_fact</code>
glm-5	Z.AI	37/40	13/40 (32%)	RAG re-retrieval; see below

GLM-5: model refusal, not defense. Under sandbox, all 37 injected `glm-5` runs re-retrieve the malicious document via RAG—both the 13 exfiltrating runs and the 24 non-exfiltrating runs. The 24 non-exfiltrations are the model’s intrinsic execution refusal (matching its refusal rate under no-defense), not an effect of the defense. Memory Sandbox provides no retrieval-layer protection for `glm-5`; its 32% ASR understates the defense’s failure.

Injection-floor confound. A separate six-model arm (no-defense vs. Memory Sandbox, $N=40$) found that removing `memory_recall_fact` from the schema suppressed `memory_save_fact` in the injection session for five of six models—a tool-schema behavioral-anchor effect in which the model’s action selection changes when an unused tool is removed. Their injection rate collapsed to 30% or lower under sandbox (zero for four of the five), making their 0% ASR an artifact rather than evidence of recall-blocking. Only `gpt-oss-120b` held injection at 100% under sandbox, where

it bypassed the defense (22/40 ASR) via benign-session re-injection (21 of 22 exfiltrations in S3). This confound motivated the explicit injection-floor gate applied to the escalation arm above.

Provider breadth. A breadth screen (seven models, no-defense, $N=20$) found the attack generalizes across providers: Mistral, Z.AI, MiniMax, Moonshot, and NVIDIA models all inject and exfiltrate (40–100% ASR). Llama 4 Maverick (Meta) was injection-resistant (0/20 injection, tagged *model_resisted_injection* in all runs)—the first non-Anthropic injection-resistant model we observe.

Evaluation artifact: Bedrock Converse tool-name rejection. Two models (gpt-oss-safeguard-120b, gpt-oss-20b) intermittently emitted tool-call names violating the Bedrock Converse name regex `[a-zA-Z0-9_-]+` in the single-shot no-attack context, causing the request to be rejected and the run to terminate with empty logs. We reclassified 17 such runs (of 977 Bedrock validation records) as infrastructure errors and excluded them; no-attack BPCR is 100% on all completed runs. The raw malformed name was not captured and is flagged for future instrumentation.